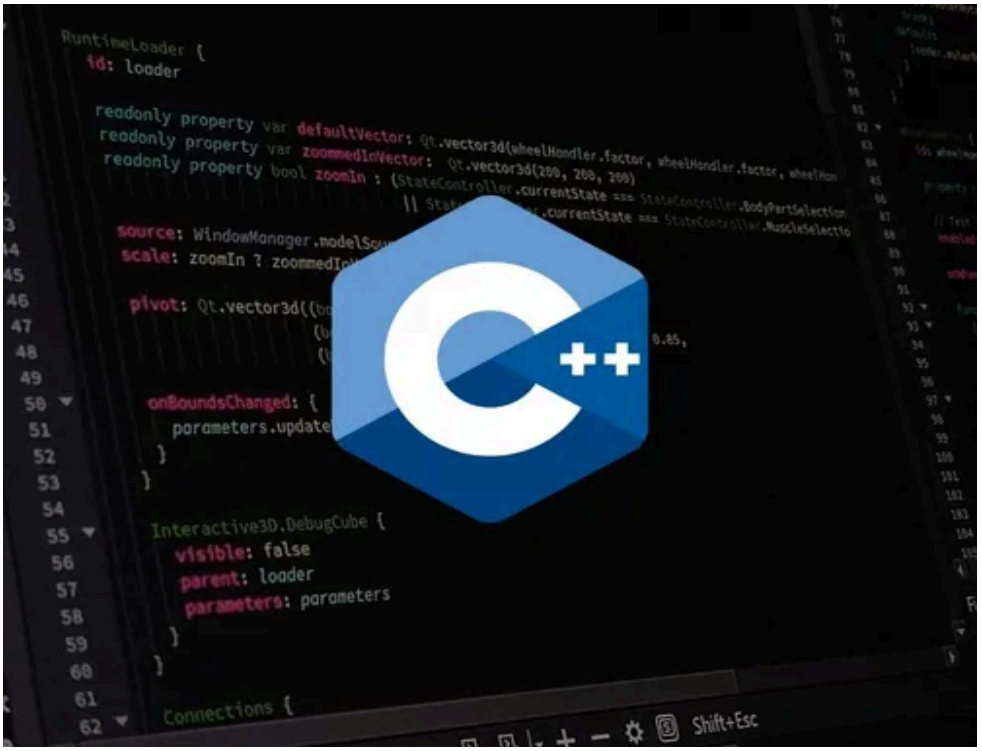
Materia: Laboratorio de Programación
EEST N.° 1 - "Eduardo Ader" Vicente López
5° año 3° Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs

1° CUATRIMESTRE
📺🛠️🖌️ ACTIVIDAD 6: TALLER PRÁCTICO DE REPASO Y CONSOLIDACIÓN EN C++ - LPR 5° 3° A-B



Estudiante: Mikaela Batirola
Curso: 5° 3°
Profesores: Yamil Ganduglia y York Muñoz Mansilla

Repositorio Git Hub:
https://github.com/mikabatirola434-svg/repasogeneral

Materia: Laboratorio de Programación
EEST N.° 1 - "Eduardo Ader" Vicente López
5° año 3° Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs

Cuestionario técnico de control

1. ¿Qué error crítico ocurre en la memoria RAM si una función recursiva carece de un caso base de corte?

Si una función recursiva no tiene un caso base, se sigue llamando a sí misma sin detenerse. Esto hace que se acumulen cada vez más llamadas en la memoria hasta quedarse sin espacio en la pila, provocando un **Stack Overflow**, que significa que la pila de memoria se quedó sin espacio disponible para seguir guardando nuevas llamadas de la función.

2. ¿Por qué la instrucción break optimiza el rendimiento del microprocesador al realizar una búsqueda en arreglos?

El uso de **`break`** mejora la eficiencia porque permite detener la búsqueda apenas se encuentra el valor que estamos buscando. Así, el programa no tiene que seguir recorriendo el resto del arreglo y se ahorran operaciones innecesarias.

3. ¿Cómo comprobar mediante el operador & que los elementos de un array ocupan celdas contiguas en la RAM?

Podemos comprobar que los elementos del arreglo están en posiciones consecutivas de memoria usando **&** para mostrar la dirección de cada elemento. Al comparar esas direcciones, podemos observar que van aumentando de forma consecutiva, porque los elementos de un arreglo se almacenan juntos en memoria.

4. ¿Qué función cumple la variable temporal en el algoritmo de intercambio de valores con punteros?

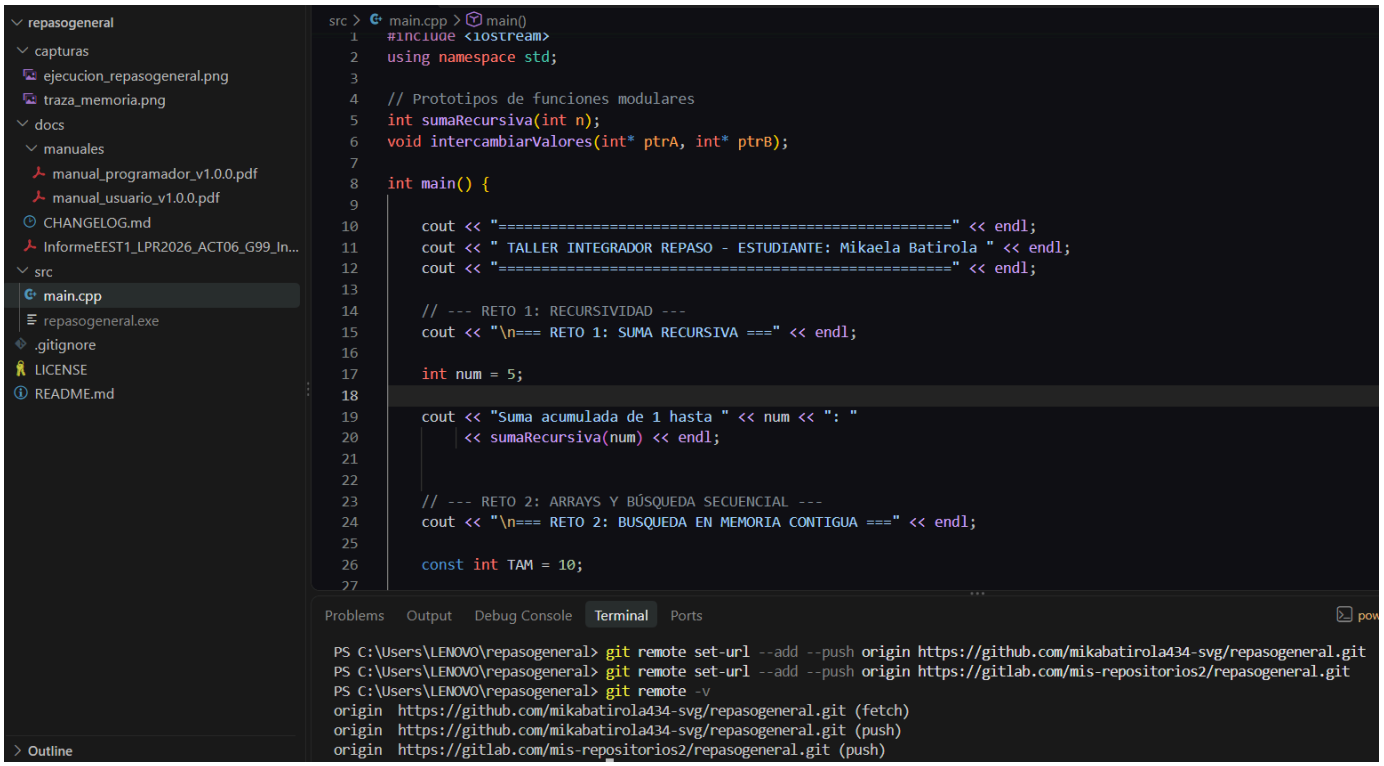
La variable temporal sirve para guardar uno de los valores mientras se realiza el intercambio. Por ejemplo, primero guarda el valor de **`ptrA`**, después se coloca el valor de **`ptrB`** en **`ptrA`** y finalmente se usa la variable temporal para colocar el valor original de **`ptrA`** en **`ptrB`**. Así no se pierde ninguno de los valores.

Materia: Laboratorio de Programación
EEST N.° 1 - "Eduardo Ader" Vicente López
5° año 3° Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs

5. Autoevaluación

El reto que me resultó más difícil fue el de los punteros y el intercambio de valores, porque al principio puede ser confuso entender las direcciones de memoria y el uso de * y **&**. Lo pude resolver entendiendo que **&** permite obtener la dirección de una variable y * permite acceder al valor que se encuentra en esa dirección. Después, al probar el programa, pude comprobar que los valores se intercambiaban correctamente.

Figura 1: Captura de la terminal ejecutando `git remote -v` mostrando claramente que Git está configurado para subir (push) a GitHub y a GitLab.



Materia: Laboratorio de Programación
EEST N.° 1 - "Eduardo Ader" Vicente López
5° año 3° Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs

Captura de la terminal con la ejecución del archivo `repasogeneral.exe`

